



Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Candidate:
Pierpaolo Spadafora

Supervisors at the University of Potsdam:
Prof. Thorsten Schaub
Dr. Javier Romero

Student ID 231053

Contents

Abstract	ii
1 Introduction	1
1.1 Context	1
1.2 Grounding Bottleneck and Dynamic Heuristics	1
1.3 Objective	2
1.4 Contributions	2
1.5 Structure of the Thesis	2
2 Methodology	4
2.1 System Architecture	4
2.2 Internal Syntax for Lazy Heuristics	4
2.3 Runtime Representation	5
2.4 Parsing of <code>__heuristic</code> Facts	6
2.5 Dynamic Aggregates	6
2.6 Candidate Materialization	7
2.7 Candidate Evaluation	7
2.8 Local Resolution and Global Ranking	8
2.9 Search-Time Behaviour	9
2.10 ASP Converter	9
2.11 BSP Benchmark	9
3 Results	12
3.1 Functional Validation	12
3.2 Experimental Dataset	12
3.3 Reduction of the Ground Heuristic Component	13
3.4 Runtime Behaviour	14
3.5 Choices, Conflicts, and Restarts	15
3.6 Rules, Variables, and Memory	17
3.7 Direct and Auxiliary Forms	19
3.8 Effect of the Constraint Formulation	19
3.9 Summary of Results	19
4 Discussion and Conclusions	21
4.1 Discussion of the Results	21
4.2 Limitations	21
4.3 Implications	22
4.4 Future Work	22
4.5 Conclusions	22

Abstract

Answer Set Programming (ASP) provides a declarative way to describe combinatorial problems while delegating the construction and exploration of the solution space to a grounding and solving system. In ground-and-solve systems such as clingo, the relevant part of a program is instantiated before search starts. This architecture is robust and mature, but it can become expensive when heuristic directives generate a large number of ground instances, especially when the intended heuristic depends on the partial state of the computation.

This thesis presents the design, implementation, and evaluation of a lazy heuristic propagator integrated into a modified version of clingo. The work introduces a compact declarative representation based on `__heuristic(...)` facts. These facts are read by the propagator during initialization and evaluated during search, thereby avoiding the full materialization of large families of `#heuristic` directives. The implementation aims to preserve an operational behaviour close to clingo for the modifiers `level`, `sign`, `true`, and `false`, while making explicit the difference between Alpha-style lazy evaluation and clingo-like evaluation of negative bodies and aggregates.

The main contributions are a C++ extension of clingo built around `Clingo::Heuristic`, a parser and runtime representation for lazy heuristic templates, incremental support for dynamic aggregates, a Python converter from native `#heuristic` directives to lazy and auxiliary variants, and an experimental suite for Balanced Set Partitioning (BSP). The suite includes semantic validation, benchmark execution, metric collection, and automatic graph generation.

The results show that, on the BSP benchmark, the lazy representation drastically reduces the number of ground heuristic objects. At $n = 100$, the native variants produce on average 1,010,200 ground `#heuristic` directives, whereas the direct lazy variants keep only two `__heuristic` facts. This does not remove the grounding cost of the base program, but it compresses the heuristic component and separates the growth of the problem encoding from the runtime evaluation of the search guidance.

1 Introduction

1.1 Context

Answer Set Programming (ASP) is a declarative paradigm for knowledge representation and combinatorial problem solving. Instead of prescribing an algorithm step by step, the modeller describes the properties that admissible solutions must satisfy. The ASP system then computes the answer sets, that is, the stable models of the program, which correspond to the solutions of the encoded problem [1, 2].

Most modern ASP systems follow a *ground-and-solve* architecture. A grounder first rewrites the non-ground program, which may contain variables, into a propositional program. A solver based on CDCL-style techniques then explores assignments and computes stable models [2]. *clingo* embodies this architecture by combining *gringo*, the grounder, and *clasp*, the solver. The separation is effective in many domains, but it also exposes a well-known limitation: grounding may become the bottleneck whenever the propositional instance grows much faster than the original declarative specification.

Domain-specific heuristics are one of the main tools for guiding search. In *clingo* they can be written as directives of the form:

```
1 #heuristic A : Body. [W@P, M]
```

Here A is the target atom, $Body$ is the applicability condition, W is the heuristic weight, P is a local priority, and M is the modifier. The literature shows that domain-specific heuristics can be decisive on difficult industrial instances [3, 4]. At the same time, if a heuristic directive contains aggregates or a rich body, grounding the directive may produce a very large number of ground heuristic objects.

This thesis focuses precisely on that tension. The goal is to preserve a declarative form for heuristics, while moving part of their evaluation from grounding time to search time. The result is not a general lazy grounder for ASP. It is a specialized lazy mechanism for heuristics, implemented as a heuristic propagator inside *clingo*.

1.2 Grounding Bottleneck and Dynamic Heuristics

The grounding bottleneck appears when the propositional representation required by the solver becomes too large in time or memory. The phenomenon is especially visible in programs that combine numeric domains, aggregates, and conditions whose relevance depends on the evolving search state. Lazy-grounding research addresses this issue by interleaving grounding and solving, instantiating only the parts of the program that are needed during search [5].

Dynamic heuristics add a second layer of complexity. A useful heuristic choice may depend not only on the input instance, but also on the partial assignment currently built by the solver. A decision may become preferable because some atoms are already true, because other atoms are still undecided, or because an aggregate computed over true atoms has reached a favourable value. This idea is studied in *Alpha*, a lazy-grounding system in which declarative heuristics can reason about partial states and dynamic aggregates [5, 6].

clingo already offers a mature language for `#heuristic` directives and a Domain heuristic in the solver, but its ground-and-solve pipeline materializes the ground instances of such directives before search. When the heuristic component grows significantly, it becomes attractive to keep the heuristic in a compact representation and evaluate it at runtime.

1.3 Objective

The objective of this thesis is to design and evaluate an extension of clingo that can interpret ground facts of the following form:

```

1  __heuristic(__target(b), x, __n_c, __bind(s, __sum(c, 0)),
2           __weight(__add(__mul(__add(n, 1), s), self)),
3           __priority(0),
4           __modifier(true),
5           __semantics(clingo)).

```

Such facts describe a heuristic rule compactly. During initialization, the propagator reads them, builds runtime templates, materializes heuristic candidates for the relevant ground targets, and re-evaluates those candidates whenever the solver assigns or unassigns observed literals. In this way, the number of heuristic facts in the grounded program can remain constant even when the equivalent native directive would generate hundreds of thousands or millions of ground directives.

The thesis addresses three questions. First, it asks how a `#heuristic` directive can be represented in a compact ground form that remains readable by a propagator. Second, it studies how far the clingo-like semantics of weights, local priorities, and modifiers can be reproduced through the `Clingo::Heuristic` interface. Third, it evaluates the impact of the lazy representation on grounding and search in the BSP benchmark.

1.4 Contributions

The implemented system consists of a new C++ component in `clingo-modified/libclingo`, including shared types, a parser, aggregate states, and the heuristic propagator itself. The propagator is automatically registered in `clingo_app.cc`, so that the modified clingo binary can use the mechanism without requiring an external Python controller. The project also includes the converter `test_folder/tools/asp_heuristic_converter.py`, which translates selected `#heuristic` directives into lazy facts, and a BSP experimental suite under `test_folder/benchmarks`. The suite collects grounding and solving metrics, validates the relevant answer sets, and generates the figures used in the evaluation. A step-by-step account of the current semantics is kept in `test_folder/esecuzione_passo_passo.txt`.

1.5 Structure of the Thesis

Section 2 describes the methodology: the system architecture, the internal syntax, the parser, the propagator, the converter, and the benchmark design. Section 3

presents the experimental results on BSP and discusses the main metrics. Section 4 concludes the work by highlighting limitations, implications, and possible directions for future research.

2 Methodology

2.1 System Architecture

The project extends a local copy of `clingo`, stored in `clingo-modified`, with a component dedicated to lazy declarative heuristics. The extension separates three concerns that are normally intertwined in native `#heuristic` grounding: the declarative description of the heuristic inside the ASP program, the parsing of this description into internal C++ structures, and the incremental evaluation of heuristic effects during search through the `Clingo::Heuristic` interface.

An input program contains the ordinary rules of the problem and one or more `__heuristic(...)` facts. `clingo` still grounds the program in the standard way, but during the propagator initialization phase the extension scans the symbolic atoms for `__heuristic` facts, parses them into `HeuristicRuleTemplate` objects, indexes the ground literals used by targets and bodies, and materializes runtime heuristic candidates for the relevant ground targets. During search, the solver calls `propagate`, `undo`, and `decide`; the propagator updates the active heuristic effects and, when possible, returns the preferred decision literal.

The extension is registered in `clingo-modified/libclingo/src/clingo_app.cc`. Inside `ClingoApp::run`, the modified binary creates the propagator and attaches it to the `Control` object:

```
1  Clingo::Control ctl(reinterpret_cast<clingo_control_t*>(grd.get()), false);
2  HeuristicPropagator my_propagator;
3  ctl.register_propagator(my_propagator, true);
```

The second argument requests sequential registration. This is consistent with the current implementation, which keeps mutable ranking and aggregate state inside the propagator and does not yet provide a parallel synchronization layer.

2.2 Internal Syntax for Lazy Heuristics

The native `clingo` form is:

```
1  #heuristic A : Body. [W@P, M]
```

The proposed representation rewrites the same information as a compact fact:

```
1  __heuristic(__target(pred),
2             positive_body,
3             __n_negative_body,
4             __bind(var, __sum(source, 0)),
5             __weight(expr),
6             __priority(expr),
7             __modifier(true),
8             __semantics(clingo)).
```

The first argument selects the target predicate. Body arguments may be implicit, as in a nullary symbol such as `x` that is matched against the target tuple, or explicit, through `__body(pred, __match(...), __bind_arg(...))`. Negative body predicates are encoded with the `__n_` prefix. Runtime variables can be bound either from target arguments through `__bind_target_arg`, from body arguments through `__bind_arg`, or from dynamic aggregates through `__bind(var, __sum(...))`. The remaining arguments describe the weight expression, the local priority expression, the modifier, and the intended evaluation semantics.

The direct clingo-like BSP heuristic used in the experiments is represented as follows:

```

1  __heuristic(__target(b), x, __n_c, __bind(s, __sum(c, 0)),
2           __weight(__add(__mul(__add(n, 1), s), self)),
3           __priority(0),
4           __modifier(true),
5           __semantics(clingo)).

```

Conceptually, this corresponds to the native directive:

```

1  #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}.
2  [(((n+1)*S)+X)@0, true]

```

The important semantic point is that `__priority` is not used as a global ranking key among different targets. In clingo, the priority `@P` resolves conflicts among applicable modifications for the same target and modifier. The global ranking among targets is instead induced by the resolved level after modifiers have been applied.

2.3 Runtime Representation

The shared data structures are defined in `clingo-modified/libclingo/clingo/heuristic_types.hh`. The central type is `HeuristicRuleTemplate`:

```

1  struct HeuristicRuleTemplate {
2      Clingo::Symbol target_pred;
3      std::vector<BodyPredicateSpec> pos_body_preds;
4      std::vector<Clingo::Symbol> neg_body_preds;
5      HeuristicModifier modifier = HeuristicModifier::True;
6      HeuristicSemantics semantics = HeuristicSemantics::Clingo;
7      std::unordered_map<Clingo::Symbol, AggregateKey> var_bindings;
8      std::unordered_set<Clingo::Symbol> body_var_names;
9      std::unordered_set<Clingo::Symbol> target_var_names;
10     std::vector<TargetArgBinding> target_arg_bindings;
11     ArithmeticExpression bias_expr = ArithmeticExpression::number(0);
12     ArithmeticExpression local_priority_expr =
13         ↪ ArithmeticExpression::number(0);
14 };

```


A template is still parametric. It becomes concrete only after the propagator has indexed the ground atoms and matched the template against possible target tuples. Each `RuntimeHeuristicCandidate` stores the target literal, the target tuple values, the literals appearing in the positive and negative bodies, the values bound from body predicates, and the aggregate bindings that must be read during evaluation:

```

1 struct RuntimeHeuristicCandidate {
2     size_t rule_idx = 0;
3     Clingo::literal_t target_lit = 0;
4     int self_value = 0;
5     std::vector<int> tuple_values;
6     std::unordered_map<Clingo::Symbol, int> body_var_values;
7     std::vector<Clingo::literal_t> pos_body_lits;
8     std::vector<Clingo::literal_t> neg_body_lits;
9     std::vector<CandidateAggregateBinding> aggregate_bindings;
10 };

```

When evaluated, a candidate produces a local effect rather than a global decision order. The effect records whether the candidate is active, the target literal, the computed bias, the computed local priority, the modifier, and the candidate identifier. Active effects are then resolved per target in a `TargetHeuristicState`. Only after this local resolution does a target enter the global decision ranking.

2.4 Parsing of `__heuristic` Facts

The parser is implemented in `clingo-modified/libclingo/src/heuristic_parser.cc`. Its public entry point,

```

1 std::vector<HeuristicRuleTemplate>
2 parse_lazy_heuristic_templates(std::vector<Clingo::Symbol> const
   ↪ &heuristic_symbols)

```

receives the `__heuristic` symbols already produced by grounding. For each symbol, the parser checks that a target is present, recognizes implicit and explicit positive bodies, translates negative bodies marked by `__n_`, validates aggregate bindings and target/body variables, reads `__weight` and `__priority` as arithmetic expressions, and rejects duplicated or ambiguous syntax early.

Arithmetic expressions are represented as a small AST. The current language supports integer constants, `self`, bound variables, and the explicit operators `__add`, `__sub`, and `__mul`. This avoids reparsing textual expressions during search: once the AST has been built, runtime evaluation is a recursive visit over a small and typed structure.

2.5 Dynamic Aggregates

Dynamic aggregates are implemented in `clingo-modified/libclingo/clingo/heuristic_aggregate.hh`. All aggregate states share a minimal interface with

`add`, `remove`, `result`, and `reset` operations. The current implementation provides incremental support for `sum`, `count`, `min`, and `max`; minima and maxima are maintained with ordered multisets based on `std::map`.

Each aggregate source contributes to a `RuntimeAggregateKey`. The key stores the aggregate descriptor and the instantiated filter values. Filters make it possible to express target-specific aggregates, for example:

```
1  __bind(s, __sum(source, 1, __filter(0, 0)))
```

In this case, the aggregate associated with a target such as `choose(X)` only considers source atoms `source(X, V)` whose first argument matches the first target argument.

2.6 Candidate Materialization

During initialization, the propagator builds an index from predicate names to ground literals:

```
1  using GroundLiteralIndex =
2      std::unordered_map<Clingo::Symbol, LitByTuple>;
```

The outer key is the predicate name and the inner key is the tuple of numeric arguments. The implementation currently considers only atoms whose arguments can be represented numerically. This restriction simplifies the evaluation of `self`, variable bindings, and aggregate filters, but it is also one of the present limitations of the prototype.

For each compatible ground target, the propagator collects the positive and negative body literals, applies any explicit `__match` mappings, stores values introduced by `__bind_arg`, creates a `RuntimeHeuristicCandidate`, and registers watches on the literals that may change the candidate's activation. This process is intentionally different from grounding native `#heuristic` directives. Native directives become many printed ground heuristic objects; the lazy propagator keeps only a few descriptive facts in the ground program and stores the activation logic in C++ data structures.

2.7 Candidate Evaluation

The central evaluation routine is:

```
1  bool HeuristicPropagator::evaluate_candidate_effect(
2      size_t candidate_id,
3      Clingo::Assignment const &assignment,
4      CandidateHeuristicEffect &effect
5  ) const
```

A candidate is active only when its target is still free, all positive body literals are true, the negative body literals are satisfied according to the selected semantics,

the required aggregates are available, and both the weight and priority expressions can be evaluated. The distinction between Alpha-style and clingo-like semantics is concentrated in the treatment of negative bodies and aggregates. In Alpha-style semantics, `not a` is satisfied as soon as `a` is not true; in clingo-like semantics, it is satisfied only when `a` is false. For clingo-like aggregate evaluation, the implementation can also require the relevant sources to be determined.

If the computed local priority is negative, the propagator raises an error. The implementation deliberately avoids defining an ordering with negative priorities, because such a behaviour would not be documented by the current prototype.

2.8 Local Resolution and Global Ranking

The most delicate part of the implementation is the interpretation of `@P`. Earlier versions of the prototype used the pair “priority, weight” as a global key among candidates. That choice was natural for some Alpha-style experiments, but it was not faithful to clingo. In the current design, `__priority(P)` resolves modifications that apply to the same target, while `__weight(W)` contributes to the target level or sign according to the modifier.

Modifier	Effect in the propagator
<code>level</code>	Updates the target level with value W .
<code>sign</code>	Updates only the preferred sign, using <code>sign(W)</code> .
<code>true</code>	Sets level W and a positive preferred sign.
<code>false</code>	Sets level W and a negative preferred sign.
<code>init, factor</code>	Recognized by the parser but not implemented.

Table 1: Implemented interpretation of clingo heuristic modifiers.

`init` and `factor` are intentionally not treated as aliases for `level`. In native clingo they modify the internal score of the solver heuristic; the `Clingo::Heuristic::decide` interface used here can return a decision literal, but it cannot directly update the score of the underlying heuristic. Mapping these modifiers to `level` would be easy to implement but semantically misleading.

After local resolution, targets are stored in:

```
1 std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;
```

The ranking key contains only a resolved level and a target literal:

```
1 struct DecisionRankKey {
2     int level = 0;
3     Clingo::literal_t target_lit = 0;
4 };
```

The comparison orders targets by decreasing level and then by increasing literal as a deterministic tie-breaker. This tie-breaker does not attempt to reproduce the full internal heuristic state of clingo when several targets share the same level.

2.9 Search-Time Behaviour

The propagator reacts to search events through `propagate`, `undo`, and `decide`. When an observed literal becomes true, `propagate` adds its contribution to the relevant aggregate states, refreshes candidates depending on those aggregates or on the literal itself, and updates the resolved state of affected targets. During backtracking, `undo` removes aggregate contributions and uses `noexcept` refresh routines, because exceptions during undo would violate the solver’s robustness expectations.

The `decide` method reads the currently best target from `active_decision_ranks_`, discards stale or already assigned targets, applies the resolved sign, and returns the selected literal. If no lazy target with positive level is available, the method checks whether the solver fallback literal has an active lazy sign; otherwise it returns 0, which leaves the decision to the standard solver heuristic.

2.10 ASP Converter

The converter in `test_folder/tools/asp_heuristic_converter.py` automates the rewriting of selected native `#heuristic` directives. It relies on `clingo.ast` to read the source program and emits direct lazy forms, clingo-like lazy forms, auxiliary-predicate forms, and mixed auxiliary/lazy forms. The converter recognizes targets, positive and negative bodies, aggregates, weights, priorities, and modifiers. When a positive body does not share the target tuple directly, it emits an explicit representation with `__body`, `__match`, and `__bind_arg`, as in the following example:

```
1 h_b(X,S) :- x(X), not c(X), S = #sum{Y : c(Y)}.
2
3 __heuristic(__target(b),
4             __body(h_b, __match(0, 0), __bind_arg(s, 1)),
5             __weight(__add(__mul(__add(n, 1), s), self)),
6             __priority(0),
7             __modifier(true),
8             __semantics(alpha)).
```

This form is useful when the heuristic value is already materialized by an auxiliary predicate. It also makes clear why auxiliary formulations must be evaluated separately from direct lazy formulations: they may reintroduce a large grounding cost before the propagator is even used.

2.11 BSP Benchmark

The main experimental suite is based on Balanced Set Partitioning (BSP). Given the domain $1, \dots, n$, the problem assigns each number to one of two sets, represented by the predicates `b(X)` and `c(X)`, while requiring the absolute difference between the sums of the two sets to be at most one.

The variants are separated so that three effects do not get conflated: the cost of grounding heuristic directives, the semantics used to activate heuristic information, and the formulation of the BSP constraint.

Variant	Role in the evaluation
<code>gc_noheur</code>	Ground-and-solve clingo without domain-specific heuristics.
<code>gc</code>	Native clingo-like <code>#heuristic</code> directives.
<code>ga</code>	Ground-and-solve Alpha-style activation, obtained by removing the negative body from the directive.
<code>ga_dyn</code>	Ground-and-solve reformulation with an intermediate bound <code>S <= #sum</code> ; treated as a separate modelling variant.
<code>la</code>	Direct lazy form with Alpha-style semantics.
<code>lc</code>	Direct lazy form with clingo-like semantics.
<code>la_aux</code>	Lazy Alpha form driven by auxiliary predicates.
<code>la_co</code>	Lazy Alpha form combined with a more propagation-friendly BSP constraint.

Table 2: BSP variants used in the experimental suite.

The native clingo-like variant is:

```

1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}.
2   [(((n+1)*S)+X)@0, true]
3 #heuristic c(X) : x(X), not b(X), S = #sum{Y : b(Y)}.
4   [(((n+1)*S)+X)@0, true]
```

The aggregate in the heuristic value forces the grounder to produce many directive instances. The `ga` variant removes `not c(X)` and `not b(X)` from the heuristic body:

```

1 #heuristic b(X) : x(X), S = #sum{Y : c(Y)}. [...]
2 #heuristic c(X) : x(X), S = #sum{Y : b(Y)}. [...]
```

This is not merely a faster or slower version of `gc`. It changes the moment at which the heuristic can be applied, because it no longer requires the negative body to hold. The `ga_dyn` variant is different again:

```

1 sum_value(0..tot).
2 #heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)}. [...]
3 #heuristic c(X) : x(X), sum_value(S), S <= #sum{Y : b(Y)}. [...]
```

The intermediate bound can make heuristic information applicable with partial aggregate information, so `ga_dyn` is not an equivalent baseline for `ga`. Similarly, `la_co` changes the BSP constraint itself:

```

1 :- not min_val <= #sum{Y : b(Y)} <= max_val.
```

It is therefore discussed as a modelling experiment rather than as direct evidence that lazy heuristics alone are faster.

The benchmark is orchestrated by `test_folder/benchmarks/benchmark_bsp.sh`. Each run is handled by `test_folder/benchmarks/benchmark_runner.py`, which executes `clingo` in JSON mode with `--outf=2 --stats=2`, enables `--heuristic=Domain`, applies timeout and memory limits, collects search and grounding-related statistics, runs `clingo` again with `--text` to count ground lines and heuristic objects, writes the CSV row, and stores metadata in `test_folder/results/run_metadata.json`.

Metric	Meaning
<code>ground_heuristics</code>	Number of printed <code>#heuristic</code> lines in the textual ground program.
<code>ground_lazy_heuristic_facts</code>	Number of <code>__heuristic</code> facts.
<code>ground_lines</code>	Total number of lines in the textual ground program.
<code>choices, conflicts, restarts</code>	Search behaviour reported by <code>clingo</code> .
<code>total_s</code>	The <code>Time.Total</code> field from <code>clingo</code> JSON statistics.
<code>solving_s</code>	The <code>Time.Solve</code> field.
<code>grounding_s</code>	Derived estimate <code>Time.Total - Time.Solve</code> , truncated to zero after rounding.

Table 3: Main metrics collected by the BSP runner.

The `grounding_s` value is not an independent timer for the grounder alone. It is used throughout the thesis as an estimate of the part of the run not attributed to solving in `clingo`'s statistics. The graphs are generated by `test_folder/tools/gen_graphs.py` and `test_folder/generate_graphs.sh`, and are stored under `test_folder/results/graphs/bsp/`.

3 Results

3.1 Functional Validation

The system was validated by keeping three levels separate. The first level concerns the ASP program itself: the relevant answer sets, the `SAT/UNSAT` result, and the visible solutions must remain consistent across the compared encodings. The second level concerns the heuristic transformation: targets, bodies, weights, priorities, modifiers, aggregate variables, and target variables must be preserved when a native `#heuristic` directive is rewritten as `__heuristic(...)` facts. The third level concerns the effect on search, measured through choices, conflicts, solving time, grounding estimates, and the number of ground heuristic objects.

Heuristics do not change the semantics of the ASP program: they guide search, but they do not add or remove answer sets. For this reason, semantic validation compares models after filtering out the internal prototype symbols that start with `__`, unless a complete comparison is explicitly requested with `--keep-internal`. This behaviour is implemented in `test_folder/tools/validate_lazy_semantics.py`. Performance metrics are treated separately, because they measure the impact of the heuristic representation on grounding and search rather than on the set of solutions.

The validation suite includes C++ tests in `clingo-modified/libclingo/tests/heuristic_propagator.cc`, corner-case instances in `test_folder/instances/corner_cases`, and the BSP benchmark. The C++ tests cover incremental aggregates, empty domains, unsatisfiable instances, `self` as the first target argument, explicit target and body bindings, target-specific aggregate filters, local priority resolution, and syntax errors such as duplicated variables or negative indices. These tests are important because they isolate the correctness of the heuristic machinery from the size and modelling details of the benchmark.

3.2 Experimental Dataset

The BSP run used for the graphs contains 120 rows: 12 values of n , from 10 to 120, 6 variants, and 2 seeds, except where variants stop earlier because of timeouts. The main comparison includes `gc_noheur`, `gc`, `ga`, `la`, `lc`, and `la_aux`. The variants `ga_dyn` and `la_co` are treated as methodologically separate experiments because they change either the activation condition or the BSP constraint formulation.

Variant	Maximum n solved with status ok
<code>gc_noheur</code>	110
<code>gc</code>	100
<code>ga</code>	100
<code>la</code>	110
<code>lc</code>	100
<code>la_aux</code>	50

Table 4: Largest BSP instance solved correctly in the considered run.

The first timeouts occur at $n = 60$ for `la_aux`, at $n = 110$ for `gc`, `ga`, and `lc`, and at $n = 120$ for `gc_noheur` and `la`.

3.3 Reduction of the Ground Heuristic Component

The clearest result concerns the size of the heuristic component in the grounded program. In the native `gc` and `ga` variants, the number of ground `#heuristic` directives grows rapidly with n . In the direct lazy variants `la` and `lc`, the grounded program keeps two `__heuristic` facts: one for target predicate `b` and one for target predicate `c`.

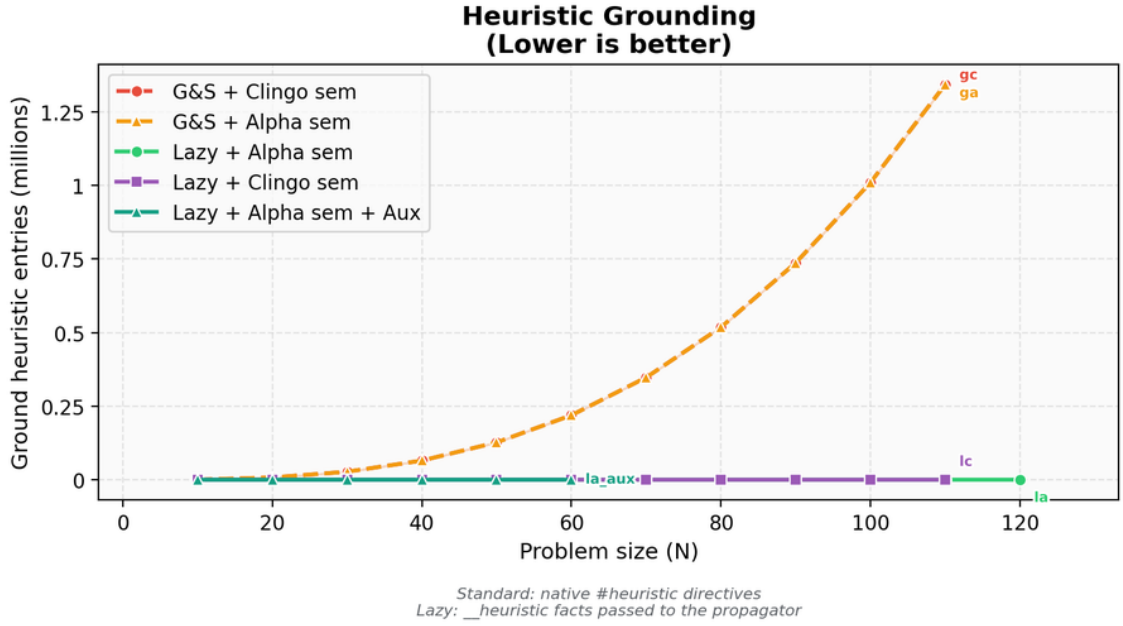


Figure 1: Comparison between native ground heuristic objects and lazy heuristic facts.

n	Native heuristic objects	Lazy facts	Ratio
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 5: Growth of the native heuristic component compared with the lazy representation.

This is the strongest evidence that the representation itself is effective. The reduction does not mean that the whole ground program becomes constant: the BSP encoding still contains rules and constraints whose size grows with n . What becomes compact is the heuristic part, which is no longer expanded into a large family of ground directives and is instead evaluated by the propagator at runtime.

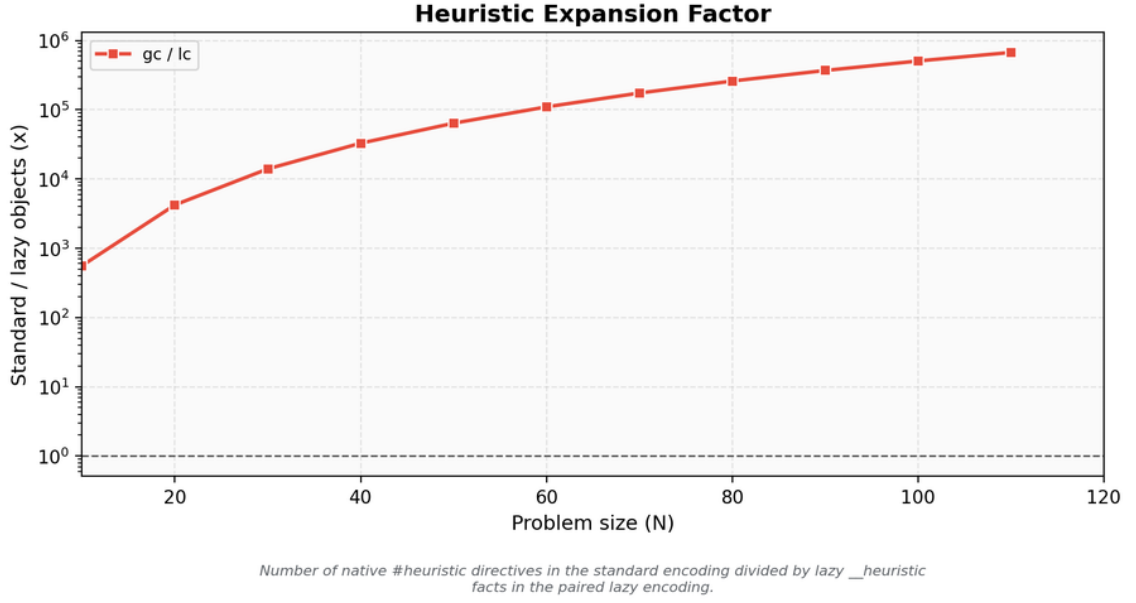


Figure 2: Expansion factor of native heuristic directives relative to lazy facts.

3.4 Runtime Behaviour

Figure 3 reports the total runtime. The decomposition of the measurements shows that, for larger values of n , the part not attributed to solving dominates the total cost. This is consistent with the BSP encoding used in the benchmark: even without native heuristic directives, the base program generates millions of ground rules.

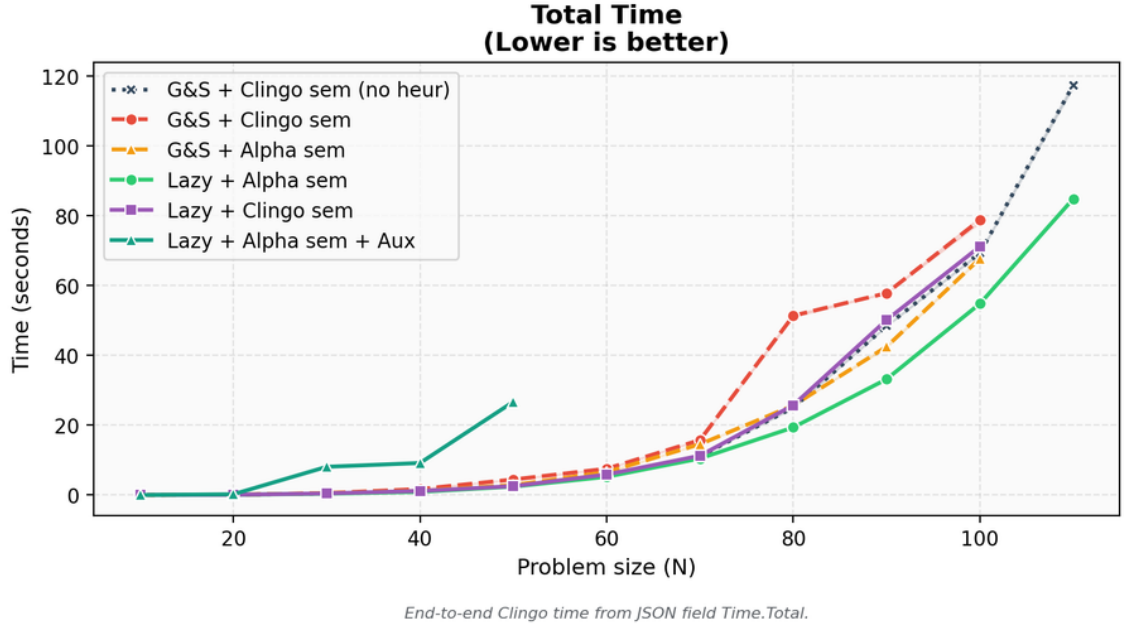


Figure 3: Total runtime for the BSP variants.

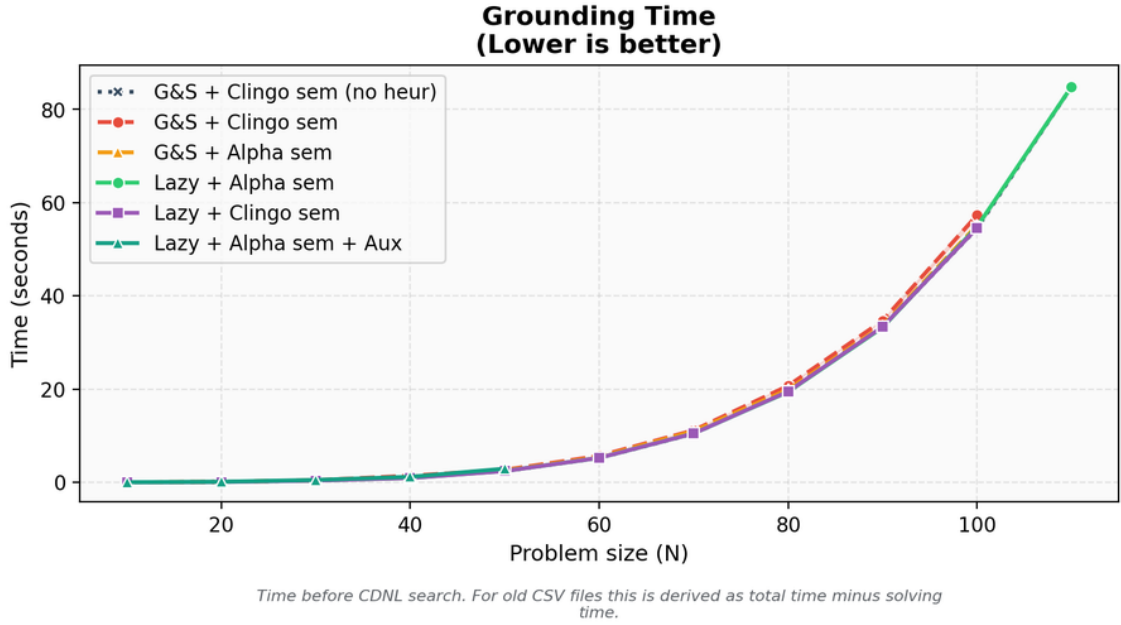


Figure 4: Grounding estimate computed as total time minus solving time in clingo JSON statistics.

The runner computes `grounding_s` as `Time.Total - Time.Solve`. It should therefore be read as a derived estimate, not as an independent timer for the grounder alone.

Variant	Total (s)	Grounding estimate (s)	Solving (s)
gc_noheur	69.40	54.49	14.91
gc	78.84	57.29	21.55
ga	67.79	55.15	12.64
1a	54.85	54.83	0.02
1c	71.23	54.58	16.65

Table 6: Average runtime at $n = 100$. The auxiliary lazy variant is excluded because it times out earlier.

At $n = 100$, `1a` has an almost negligible solving time compared with the other variants. This happens because the Alpha-style guidance leads to direct decisions and very few conflicts. The `1c` variant stays closer to the clingo-like treatment of negative bodies and therefore has a solving time comparable to `gc_noheur`, while still keeping the heuristic component compact.

3.5 Choices, Conflicts, and Restarts

Search statistics reveal a sharp distinction between Alpha-style and clingo-like semantics. The `1a` variant makes exactly n choices in the measured instances; at $n = 100$ it makes 100 choices and no conflicts. The clingo-like variants follow a profile closer to the no-heuristic baseline, even when their ground heuristic representation is smaller.

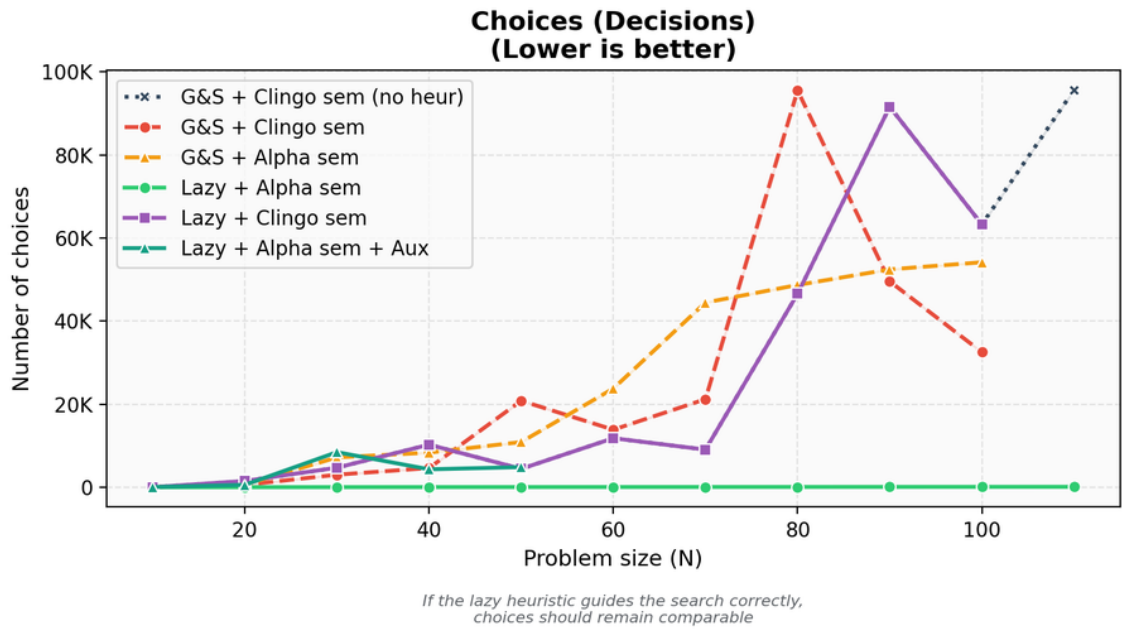


Figure 5: Number of solver choices.

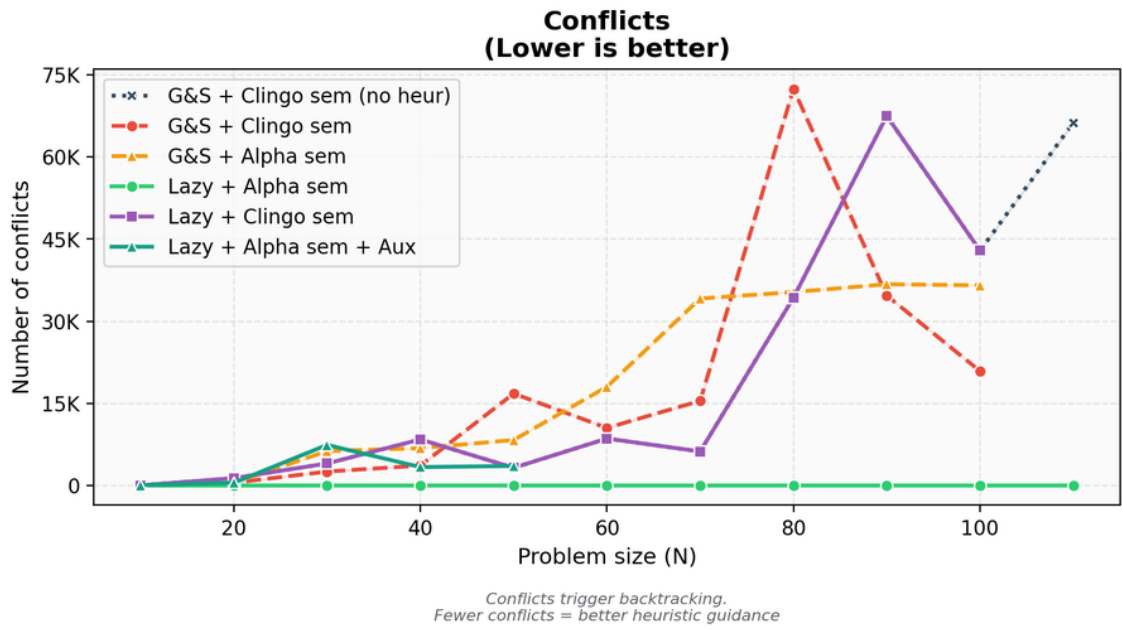


Figure 6: Number of conflicts.

Variant	Choices at $n = 100$	Conflicts at $n = 100$
gc_noheur and lc	63,255	42,903
gc	32,514	20,888
ga	54,147	36,573
la	100	0

Table 7: Average search statistics at $n = 100$.

This result must be interpreted carefully. `la` is not simply a more efficient implementation of `gc`; it uses a different activation semantics for negative bodies and aggregate availability. The thesis therefore keeps the comparison of representations separate from the comparison of semantics.

3.6 Rules, Variables, and Memory

The reduction in the heuristic component is also visible in the number of variables when native and direct lazy variants are compared. At $n = 100$, the native clingo-like variant is much larger because its grounded heuristic directives include the negative body and the aggregate in the ground-and-solve representation.

Variant	Variables at $n = 100$
gc	1,030,606
lc	20,300
ga	20,406
la	20,300

Table 8: Number of ground variables at $n = 100$.

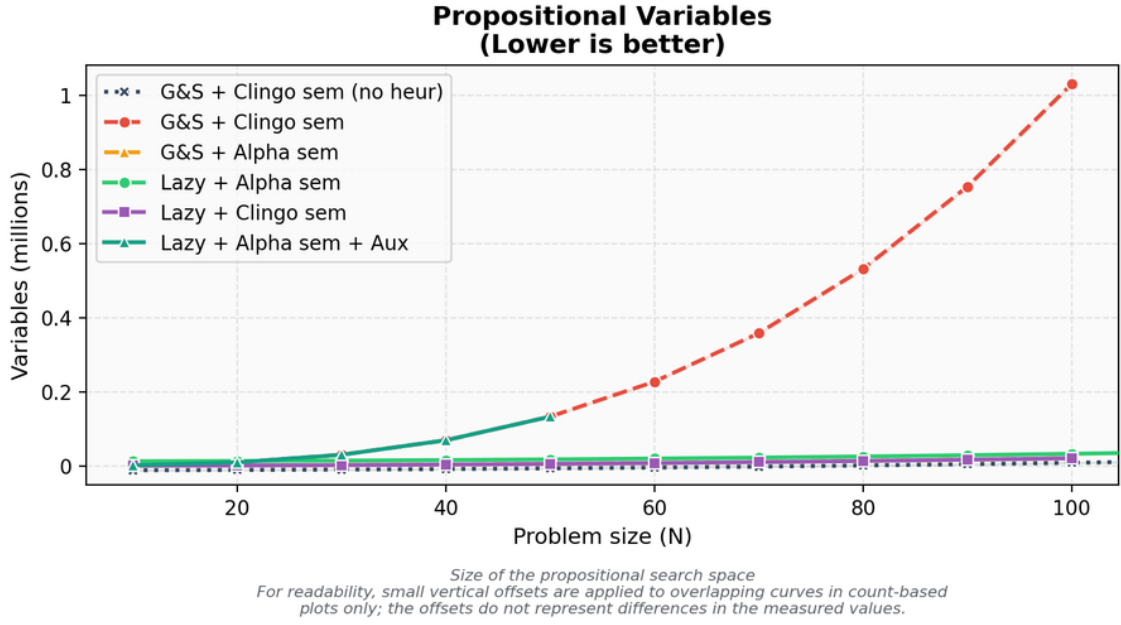


Figure 7: Number of variables in the ground problem.

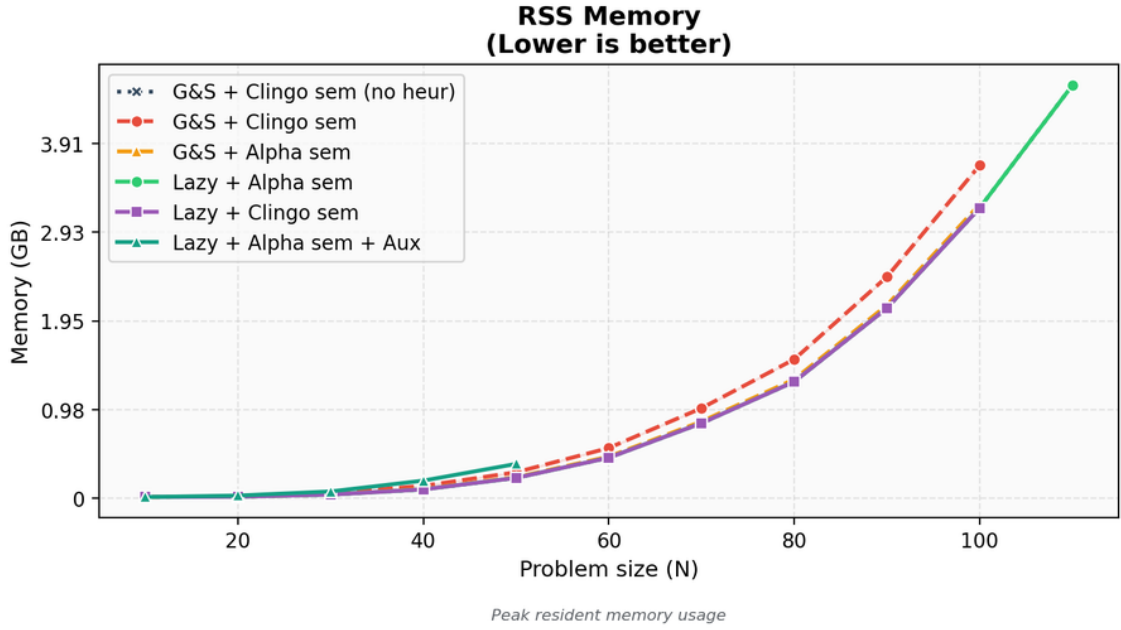


Figure 8: Maximum observed memory during the runs.

Memory is still dominated by the base program for large values of n . At $n = 100$, `gc_noheur`, `1a`, and `1c` are around 3.2 GiB, whereas `gc` reaches about 3.7 GiB. The difference confirms that native heuristics add pressure, but they are not the only source of growth.

3.7 Direct and Auxiliary Forms

The `1a_aux` variant uses auxiliary predicates to materialize the heuristic body before applying the lazy propagator. This experiment separates two effects that are otherwise easy to confuse: reducing the number of ground `#heuristic` directives, and avoiding the materialization of the heuristic body itself.

`1a_aux` keeps only two `__heuristic` facts, but it introduces `h_b` and `h_c`. The directive component is small, while the auxiliary body can become large. This explains why the variant times out already at $n = 60$ in the considered run. The direct lazy form is therefore the most interesting one for this benchmark: if the heuristic body is first materialized into a large auxiliary predicate, part of the intended benefit is lost.

3.8 Effect of the Constraint Formulation

The `1a_co` variant must be read separately from the direct comparison among heuristic representations. It changes not only the representation of the heuristic directive, but also the BSP constraint formulation by using a more propagation-friendly interval constraint. A comparison between `1a_co` and `1a` therefore measures the interaction between lazy heuristics and problem modelling, not the isolated effect of the lazy representation.

For this reason, `1a_co` is useful as an experiment on modelling choices, but it is not used as a direct baseline for claiming that a lazy heuristic is faster than an equivalent ground-and-solve heuristic.

3.9 Summary of Results

The experimental results support five conclusions. The lazy representation reduces the ground heuristic component from hundreds of thousands or millions of directives to two descriptive facts in BSP. The total runtime does not become constant because the base program is still grounded by clingo. Alpha-style semantics can guide search very aggressively, but it does not coincide with clingo-like semantics. Auxiliary forms are useful for expressing more general mappings, yet they may reintroduce the grounding cost that direct lazy evaluation tries to avoid. Finally, variants that change the BSP constraint formulation, such as `1a_co`, must be analysed as modelling experiments rather than direct comparisons among heuristic encodings.

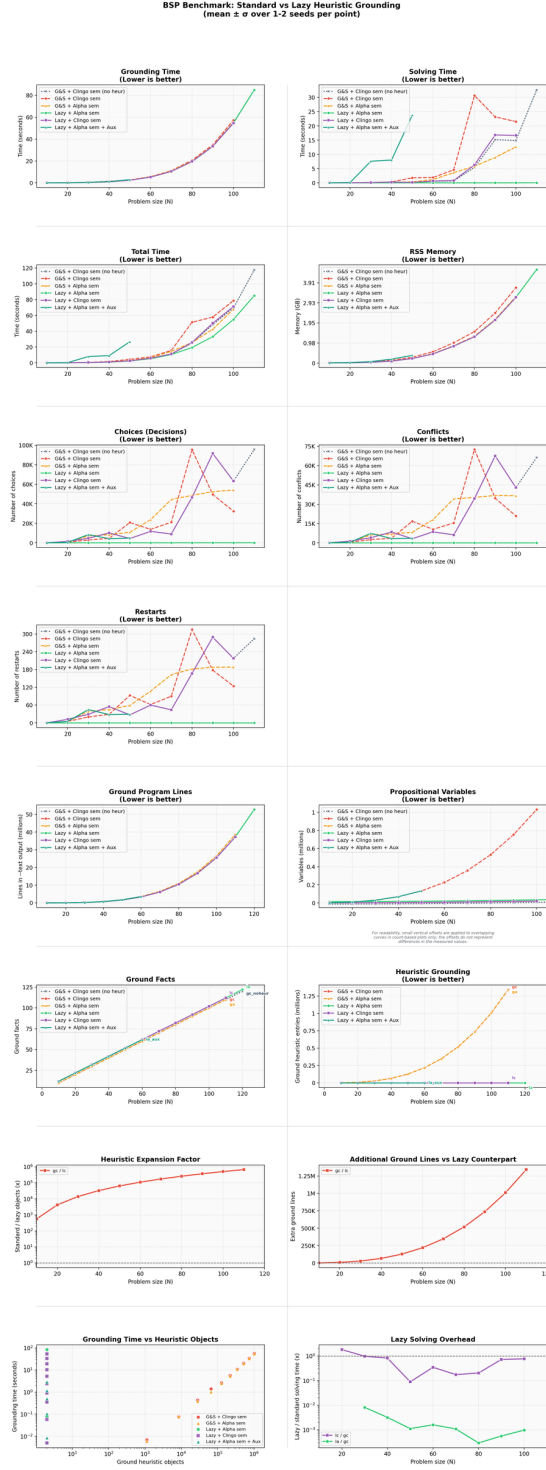


Figure 9: Overview of the BSP benchmark results.

4 Discussion and Conclusions

4.1 Discussion of the Results

The work shows that an important part of the cost of declarative heuristics can be moved from grounding to search. In the BSP benchmark, native `#heuristic` directives grow very rapidly: at $n = 100$, the native variants exceed one million ground heuristic objects. The lazy version keeps two `__heuristic` facts and delegates the evaluation of runtime candidates to the propagator.

This result should not be interpreted as a replacement for general lazy grounding. `clingo` still grounds the base program. The propagator acts only on the heuristic component, namely on the representation and evaluation of rules whose purpose is to guide search. Consequently, reducing the heuristic component does not remove the growth of the BSP rules, but it makes one specific source of growth explicit, measurable, and controllable.

The second central aspect is semantic. The thesis distinguishes `clingo`-like semantics, where negative bodies and aggregates are evaluated conservatively, from Alpha-style semantics, where non-truth may already make a negative body applicable and aggregates are read with respect to the partial state. This distinction is essential. The `1a` variant has very strong solving performance, but it does not follow the same semantics as `gc`. The `1c` variant is more appropriate for measuring the lazy representation against a `clingo`-like interpretation, even though its search behaviour remains closer to the baseline.

This semantic distinction is independent of ASP correctness. Heuristics do not change the set of answer sets; they only influence how the solver searches for them. The semantic validator therefore compares the visible models of the problem after filtering internal `__...` symbols, while the experimental metrics measure how the heuristic representation changes grounding and search. In the same spirit, variants such as `1a_co` must be interpreted as experiments on the BSP model, not as direct comparisons between heuristic representations alone.

4.2 Limitations

The current implementation has several technical and semantic limitations. It supports only targets and bodies that can be indexed through numeric tuples. Arithmetic expressions are limited to addition, subtraction, and multiplication. The available dynamic aggregates are `sum`, `count`, `min`, and `max`. The modifiers `init` and `factor` are recognized but not implemented, because native `clingo` uses them to update the internal heuristic score, whereas the current propagator interface can only return a decision literal. The tie-break between targets with the same resolved level is deterministic but does not reproduce the full internal score of the solver. Finally, the propagator is registered automatically and sequentially, and the main benchmark is currently concentrated on BSP.

The limitation on `init` and `factor` is particularly important. Treating them as `level` would make the prototype appear more complete, but it would give those modifiers a misleading semantics. The implementation therefore prefers an explicit limitation over an approximation that could invalidate comparisons with native `clingo`.

4.3 Implications

The main implication is that declarative heuristics can be treated as an intermediate layer between modelling and solving. One does not have to choose only between fully grounded `#heuristic` directives and procedural heuristics embedded directly in solver code. The `__heuristic` representation keeps a declarative surface, while the propagator evaluates it at runtime with access to the partial assignment.

From an engineering perspective, the extension also shows that clingo’s propagator and heuristic interfaces are flexible enough to prototype non-trivial search guidance mechanisms. The propagator does not add constraints. Instead, it intervenes at decision time, combining watched literals, incremental aggregate states, per-target modifier resolution, and a global ranking of currently active targets.

4.4 Future Work

Several extensions follow naturally from the current prototype. The propagator should become configurable, for example through a command-line option or by activating it only when `__heuristic` facts are present. The expression language could be extended with integer division, modulo, comparisons, and a principled treatment of non-numeric symbols through a stable internal mapping. The tie-break policy could be improved by interacting more closely with the solver fallback heuristic, where the interface allows it.

On the evaluation side, the next step is to test equivalence more systematically between native `#heuristic` directives and the clingo-like lazy form, and to move beyond BSP. Configuration, scheduling, and packing benchmarks are natural candidates, because dynamic heuristics are often meaningful in those domains. Another promising direction is a richer converter that turns more complex native directives into lazy forms, with clearer diagnostics and stronger guarantees about the transformations it applies.

4.5 Conclusions

This thesis presented an extension of clingo for the lazy evaluation of declarative heuristics. The system introduces a syntax based on `__heuristic` facts, a dedicated parser, runtime structures for heuristic candidates, incremental dynamic aggregates, and a clingo-like treatment of local priority, level, and sign.

The BSP evaluation shows that the ground heuristic component can be reduced drastically: from more than one million native directives to two lazy facts at $n = 100$. The main benefit is not only quantitative. The work separates the representation cost of the heuristic from the grounding cost of the base program, and it makes explicit the semantic difference between Alpha-style and clingo-like evaluation.

The final result is a working, documented, and measurable prototype. It does not solve the general grounding bottleneck by itself, but it provides a concrete path for bringing dynamic declarative heuristics into clingo without always paying the cost of their full ground materialization.

Bibliography

- [1] Martin Gebser et al. *Answer Set Solving in Practice*. Morgan and Claypool Publishers, 2012.
- [2] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: <https://potassco.org>.
- [3] Martin Gebser et al. “Domain-Specific Heuristics in Answer Set Programming”. In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.
- [4] Carmine Dodaro et al. “Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems”. In: *Theory and Practice of Logic Programming* (2016).
- [5] Richard Comploi-Taupe et al. “Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach”. In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114.
- [6] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. “Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems”. In: *Proceedings of the 25th International Workshop on Configuration*. 2023.

Acknowledgements

I would like to thank Prof. Carmine Dodaro for his guidance at the University of Calabria, and Prof. Thorsten Schaub and Dr. Javier Romero for their supervision and support during my Erasmus period at the University of Potsdam. Their feedback helped shape both the modelling choices and the implementation decisions behind this work.

I am also grateful to the ASP and Potassco communities. Tools such as clingo make it possible to experiment with advanced research ideas while relying on a solid, well documented, and open foundation.

Finally, I thank my family and the people who stayed close to me throughout this thesis work, for their constant support and patience during the many long debugging sessions.